

Data Preparation Session 5 Handout

Missing values and duplicates

In our pipeline, we arrived to import the data, check and fix its general structure, and arrange columns in the right way. From now on, we focus on each column one by one. At first, we look for missing values, followed by duplicates and outliers.

Missing values

Sometimes, we do not have all values. Why is it a problem? First, because we lose information. Second, all of our analysis can be biased. If values are missing at random, on average, the analysis is unbiased. When the probability of being missing varies, even simple tools like averages can be distorted.

How to deal with missing values.

1. **Detect** them.
2. If possible, **figure out** (use other sources, simple logic) to reconstruct.
3. **Interpolate**, use inferences in the data to reconstruct (we will check only simple tools of this case, regression models, simulations are part of other courses).
4. **Ignore**: continue with the missing data (if it is not important, if there is no reasonable way to reconstruct).

First, let's check a few numbers.

```
numbirds <- c(1,2,3,4/0,0/0,NA)
```

```
numbirds
```

```
[1] 1 2 3 Inf NaN NA
```

It seems that R knows well math for a division by zero (infinite), says that 0/0 is not a number. NA stands for missing value (there is no such thing in a data frame as an empty cell in Excel, here it is NA (similar to SQL's NULL)).

We can use `is.na()` to check if a value, or a vector of values is NA:

```
is.na(numbirds)
```

```
[1] FALSE FALSE FALSE FALSE TRUE TRUE
```

Not a Number and NA are NA. Important to know when something is NA, it can be a slightly different thing.

```
is.nan(numbirds)
```

```
[1] FALSE FALSE FALSE FALSE TRUE FALSE
```

NA is not NaN.

If we are here:

```
is.infinite(numbirds)
```

```
[1] FALSE FALSE FALSE TRUE FALSE FALSE
```

```
is.finite(numbirds)
```

```
[1] TRUE TRUE TRUE FALSE FALSE FALSE
```

NaN and NA are neither finite nor infinite, this conclusion is mathematically correct.

We can use `is.na` to filter missing values.

We load the inspections data again.

```
names <- c("ID", "DBAName", "AKAName", "License", "FacilityType", "Risk", "Address",  
          "City", "State", "ZIP", "InspectionDate", "InspectionType", "Results",  
          "Violations", "Latitude", "Longitude", "Location")
```

```
inspections <- read_csv('inspections.csv', col_names=names, skip=1)
```

Let's see what we have in the dataset, now, in a different way, using summary statistics (at least for numeric variables).

```
summary(inspections)
```

We have 14 inspections without License. We sort the data; in one data frame, we store inspections of restaurants without license, and the rest (with license) in another. It can be important to separate restaurants that worked without license (special case or illegal). Remember, in data analysis, you should focus on two things: the typical/tendency and the extremities.

The filter (like the WHERE in SQL) is a positive keyword, we keep the observation where the condition is true. Sorry, TRUE.

```
unlicensed <- inspections %>% filter(is.na(License))
```

```
licensed <- inspections %>% filter(!is.na(License))
```

We move to a hot topic: heating systems in houses of different age.

```
heating <- read_csv("heating.csv")  
glimpse(heating)
```

The data is not tidy, we need the usual `pivot_longer`. With the simplest formula, we have an error message of mismatch of data types.

```
heating <- heating %>%  
  pivot_longer(-Source, names_to = "age", values_to = "homes")
```

We can use a brutal force solution: use everything as character, strings always fit together.

```
heating <- heating %>%  
  pivot_longer(-Source, names_to = "age", values_to = "homes",  
    values_transform = list(homes = as.character))
```

The result:

```
glimpse(heating)  
summary(heating)
```

In the "homes" column, we should have numbers, and for the first look, we have numbers. What can be wrong, let's force numeric.

```
heating %>% mutate(homes=as.numeric(homes))
```

! NAs introduced by coercion

Beware of this warning. Some (or maybe all) of your values are destroyed. Yes, `as.numeric()` will transform everything that can be a number, but it will create NA for non-numeric data.

Find those values. Filter can be a very handsome tool to list problematic points.

```
heating %>% filter(is.na(as.numeric(homes)))
```

Source	age	homes
<chr>	<chr>	<chr>
1 Cooking stove	Under 25 years old	.
2 Cooking stove	25 to 29 years old	Z
3 Cooking stove	30 to 34 years old	.

OK, we have 3 categories, where the main heating source is cooking stove, and the houses are younger than 35 years old. We can suppose that there are no such houses, so "." is a missing value, but represents

zero (it is not missing in data analysis terms), and Z stands for zero (and not for the ant ).

Replace with zeros using a simple, a bit manual, but efficient solution. You can use the apply() function to apply the same function to many similar cases, but it is not covered in this course.

```
heating %>%
```

```
  mutate(homes=ifelse(homes==".", 0, homes)) %>%  
  mutate(homes=ifelse(homes=="Z", 0, homes)) %>%  
  mutate(homes=as.numeric(homes)) %>%  
  filter(Source=="Cooking stove")
```

Ok, it works, we apply the changes.

```
heating %>%
```

```
  mutate(homes=ifelse(homes==".", 0, homes)) %>%  
  mutate(homes=ifelse(homes=="Z", 0, homes)) %>%  
  mutate(homes=as.numeric(homes))
```

We move to an example where missing is seemingly not missing (and when handling missing data is not a technical question, but knowledge about the subject).

```
land <- read_csv("publiclands.csv")
```

Let's see summary, and some additional descriptions.

```
summary(land)  
nrow(land)  
unique(land$State)
```

Here, everything is OK with what is in the dataset. The problem is missing, I mean missing completely from the dataset. Yeah, it was an easy catch, as you know by heart how many states are in the USA. Good news, no public land in other states.

```
missing_states <- tibble(State=c('Connecticut', 'Delaware', 'Hawaii', 'Iowa', 'Maryland', 'Massachusetts',  
'New Jersey', 'Rhode Island'), PublicLandAcres=c(0,0,0,0,0,0,0,0))
```

We put them together to practice rbind (or bind_rows from tidyverse, if you want).

```
land_full <- rbind(land, missing_states)  
land_full <- bind_rows(land, missing_states)
```

```
mean(land$PublicLandAcres)  
mean(land_full$PublicLandAcres)
```

What a difference!

We can also check the impact of missing values in a technical way.

```
iris <- read_csv("dirty petal.csv")
```

What happens when we try some aggregate functions?

```
sum(iris$Petal.Length)
mean(iris$Petal.Length)
max(iris$Petal.Length)
```

NA

NA

NA

It's normal, 5 + I don't know + 6 is I don't know.

Your friend can be the na.rm argument: remove NA from the calculation.

```
sum(iris$Petal.Length, na.rm=TRUE)
mean(iris$Petal.Length, na.rm=TRUE)
max(iris$Petal.Length, na.rm=TRUE)
```

We will have to check the maximum values when we look for outliers.

Duplicated values

A specific problem that does not exist in SQL with tables in normal form.

However, two customers may have the same name. But what if they have the same email address, phone number and physical address?

technically, we have two different cases, identical rows and overlapping rows. In case of identical rows, two rows are exactly the same, and in most cases, we have to remove the duplicates. In case of overlapping rows, we have to decide if there is one or more correct record, if only one, which one is correct. No royal way, additional information may be necessary.

We work with a table about Olympic Games. It is a txt file, so it can be anything as delimiter. We try csv.

```
olympics <- read_csv("olympic games.txt")
```

We have Sidney and Tokyo duplicated. Let's remove EXACT duplicates using unique().

```
olympics <- unique(olympics)
```

Filter out the wrong record for Japan (it was planned for 2020, but Covid was stronger)

```
olympics <- olympics %>%
  filter(!(City=='Tokyo, Japan' & Year==2020))
```

Check the dataset:

```
View(olympics)
```

Another example.

```
library(readxl)
france <- read_excel("France population 2015.xlsx")
glimpse(france)
```

It seems that real data starts in cell C4. Focus on the right range:

```
france <- read_excel("France population 2015.xlsx", range = "C4:D12")
```

```
glimpse(france)
```

What is the population of France? It seems to be an addition.

```
sum(france$Population)
```

Always check categories. Now, remove total and gender breakouts

```
france <- france %>%  
  filter(!(Category %in% c('Total', 'Male', 'Female')))
```

What is the population of France?

```
sum(france$Population)
```

```
[1] 66675000
```

That's correct.

Units / Data conversion

Sometimes data is not in the right units. Kilometers and miles, kilograms and pounds, °C and °F, or simply kilograms and tons.

For a demonstration, we go back to our Mexican weather dataset. We reuse our previous code:



```
weather <- read_csv("mexicanweather.csv")  
weather <- pivot_wider(weather, names_from=element, values_from=value)  
weather
```

OK, we have the right format, but take a look at the data.

```
tail(weather)
```

It seems that we have a series of missing values. Honestly, where there is no minimum and maximum temperature, it is a useless observation. Delete rows that have two NA values.

```
weather <- weather %>%  
  filter(!(is.na(TMAX) & is.na(TMIN)))
```

Technically, it's OK, but we can make it more readable (and practice rename and select):

```
weather <- weather %>%  
  rename(maxtemp=TMAX, mintemp=TMIN) %>%  
  select(station, date, mintemp, maxtemp)
```

```
head(weather, n=20)
```

What are the units? Celsius? Too hot... Fahrenheit? Still too hot. Kelvin? Too low.
Maybe a decimal separator! Divide the temperatures by 10 to get them in degrees Celsius.

```
weather <- weather %>%  
  mutate(mintemp = mintemp/10) %>%  
  mutate(maxtemp = maxtemp/10)
```

We can also create a copy of the dataset in Fahrenheit.

```
weather_fahrenheit <- weather %>%  
  mutate(mintemp=mintemp*(9/5)+32) %>%  
  mutate(maxtemp=maxtemp*(9/5)+32)
```